

Informe de Laboratorio 06

Tema: REST vs. RESTful: Diseño e Implementación de Servicios Distribuidos

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	<i>REST vs. RESTful: Diseño e Implementación de Servicios Distribuidos</i>				
NÚMERO DE PRÁCTICA:	<i>06</i>	AÑO LECTIVO:	2026	NRO. SEMESTRE:	2026A
FECHA DE PRESENTACIÓN	<i>25/05/2026</i>	HORA DE PRESENTACIÓN	23:59		
INTEGRANTE (s): GRUPO “ Intocables ” • CHAMBILLA PERCA RICARDO • JUAN DIEGO GUTIERREZ CCAMA • RYAN FABIAN VALDIVIA SEGOVIA				NOTA:	
DOCENTE(s): <i>Mg. Maribel Molina Barriga</i>					

1. Repositorio de GitHub y Ejecución

El código fuente completo de este laboratorio se encuentra alojado en el siguiente repositorio de GitHub: https://github.com/rikich3/LAB_SISTEMAS_DISTRIBUIDOS

A continuación, se adjuntan capturas correspondientes a la ejecución de los servicios RESTful implementados durante el laboratorio.

2. Tareas Realizadas

Como parte de la sesión de laboratorio 6 de Sistemas Distribuidos, se llevaron a cabo las siguientes tareas:

- **Análisis de Conceptos REST y RESTful:** Comprensión profunda de los principios arquitectónicos REST definidos por Roy Fielding y su implementación práctica en APIs RESTful.
- **Estudio del Ejemplo Resuelto:** Análisis detallado de una API RESTful de gestión de productos implementada en Python con Flask, incluyendo operaciones CRUD.
- **Implementación de Ejercicios:** Desarrollo de dos aplicaciones RESTful completas:
 1. Biblioteca de Libros (Java + Spring Boot)
 2. Gestión de Estudiantes (Python + Flask)

- **Consumo de APIs:** Desarrollo de interfaces HTML/JavaScript y pruebas con Postman para validar el correcto funcionamiento de los endpoints.
- **Pruebas y Validación:** Ejecución completa de operaciones GET, POST, PUT y DELETE en todos los servicios implementados.
- **Análisis Comparativo:** Comparación entre diseños REST y no-RESTful, así como entre patrones RESTful y RPC.
- **Documentación:** Elaboración de este reporte con explicaciones detalladas de los conceptos y resultados obtenidos.

3. Desarrollo de los Ejercicios

3.1. Ejercicio Resuelto: API RESTful de Gestión de Productos

En esta sección se presenta la implementación de una API RESTful básica para la gestión de productos, desarrollada en Python con el framework Flask.

3.1.1. Descripción General

Esta API implementa las operaciones CRUD (Create, Read, Update, Delete) siguiendo estrictamente los principios RESTful:

- **GET /productos:** Obtiene la lista completa de productos
- **POST /productos:** Crea un nuevo producto
- **DELETE /productos/{id}:** Elimina un producto específico

3.1.2. Código Backend - Flask

Listing 1: app.py: API RESTful de Productos

```
1 from flask import Flask, jsonify, request
2
3 app = Flask(__name__)
4
5 productos = [
6     {"id": 1, "nombre": "Laptop"},
7     {"id": 2, "nombre": "Mouse"}
8 ]
9
10 # GET -> Obtener productos
11 @app.route('/productos', methods=['GET'])
12 def obtener():
13     return jsonify(productos)
14
15 # POST -> Agregar producto
16 @app.route('/productos', methods=['POST'])
17 def agregar():
18     data = request.json
19     productos.append(data)
20     return jsonify({"mensaje": "Producto agregado"}), 201
21
22 # DELETE -> Eliminar producto
```

```

23 @app.route('/productos/<int:id>', methods=['DELETE'])
24 def eliminar(id):
25     global productos
26     productos = [p for p in productos if p["id"] != id]
27     return jsonify({"mensaje": "Producto eliminado"})
28
29 if __name__ == '__main__':
30     app.run(debug=True)

```

Análisis del Código:

- Se utiliza Flask como framework web ligero
- Los datos se almacenan en una lista de diccionarios (para demostración)
- Cada endpoint utiliza el método HTTP apropiado según la operación
- Las respuestas se devuelven en formato JSON
- El código de estado 201 (Created) se utiliza para POST exitosos

3.1.3. Pruebas con Postman - GET

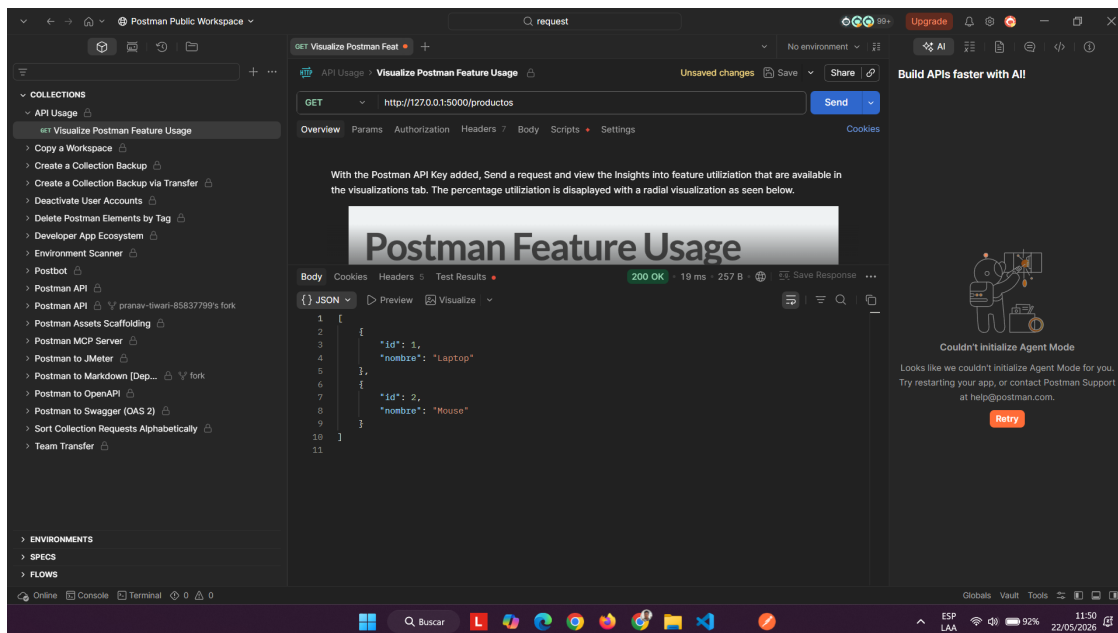


Figura 1: Prueba GET: Obtención de lista de productos

Observación: La solicitud GET a `http://localhost:8080/libros` retorna un arreglo JSON con todos los productos almacenados.

3.1.4. Pruebas con Postman - POST

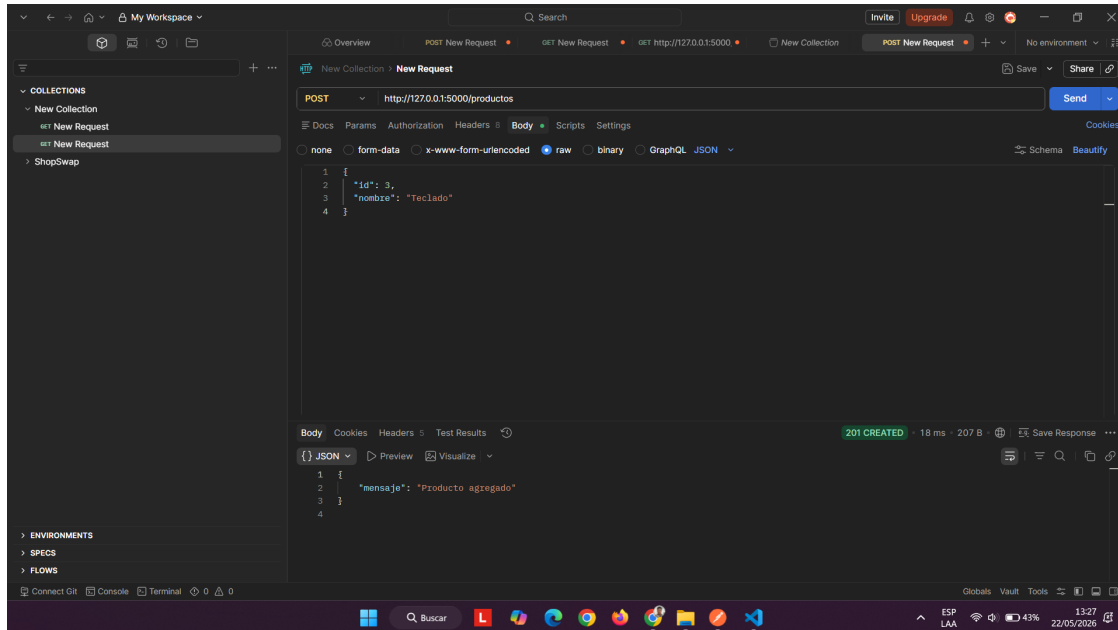


Figura 2: Prueba POST: Creación de nuevo producto

Observación: La solicitud POST enviando un JSON con los datos del producto retorna un código 201 (Created) confirmando la creación exitosa.

3.1.5. Pruebas con Postman - DELETE

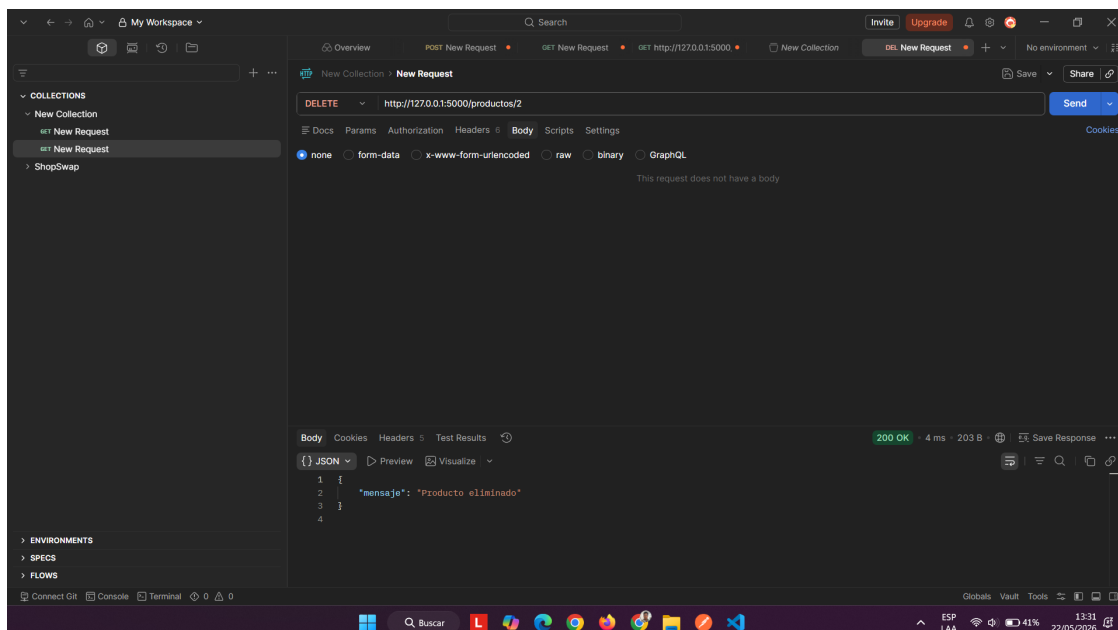
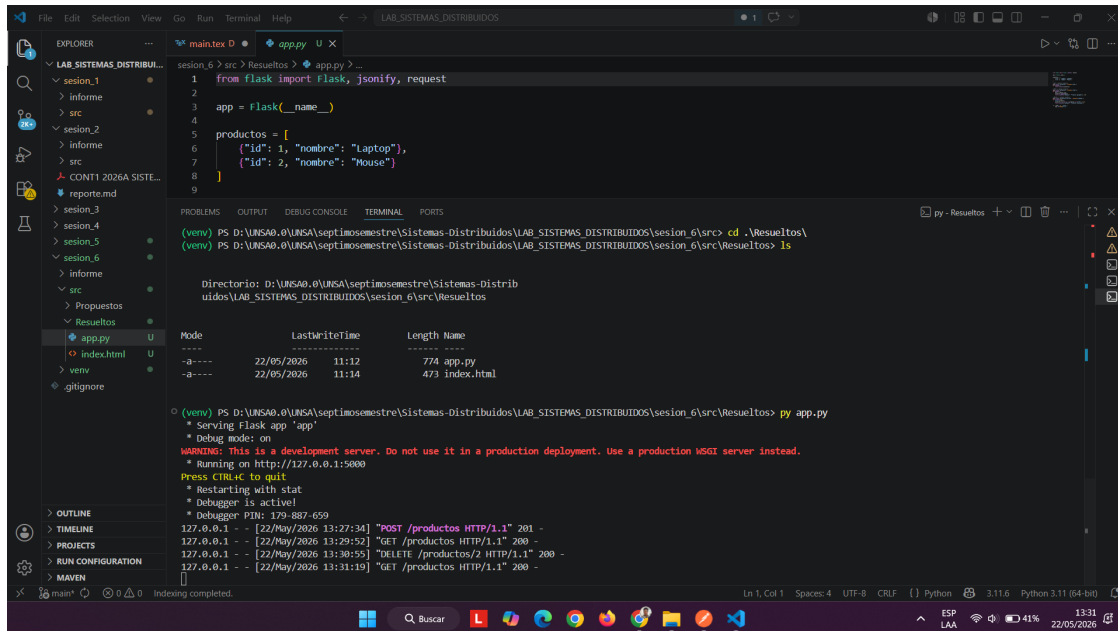


Figura 3: Prueba DELETE: Eliminación de producto

Observación: La solicitud DELETE elimina el producto especificado por ID y retorna un mensaje de confirmación.

3.1.6. Estado del Servidor



```

1 from flask import Flask, jsonify, request
2
3 app = Flask(__name__)
4
5 productos = [
6     {"id": 1, "nombre": "Laptop"},
7     {"id": 2, "nombre": "House"}
8 ]
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Directory: D:\UNSA\0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_6\src\Resueltos

Mode	LastWriteTime	Length	Name
----	-----	-----	-----
-a----	22/05/2026 11:12	774	app.py
-a----	22/05/2026 11:14	473	index.html

```

PS D:\UNSA\0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_6\src\Resueltos> py app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 129-887-659
127.0.0.1 - - [22/May/2026 13:27:34] "POST /productos HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 13:29:52] "GET /productos HTTP/1.1" 200 -
127.0.0.1 - - [22/May/2026 13:30:55] "DELETE /productos/2 HTTP/1.1" 200 -
127.0.0.1 - - [22/May/2026 13:31:19] "GET /productos HTTP/1.1" 200 -

```

Figura 4: Estado del servidor Flask mostrando todas las solicitudes procesadas

Análisis: El servidor Flask registra cada solicitud recibida con su método HTTP, ruta, código de estado y tiempo de respuesta.

3.2. Ejercicio Propuesto 1: API RESTful Biblioteca

Se implementó una API completa para gestión de libros utilizando Java con Spring Boot.

3.2.1. Interfaz de Usuario

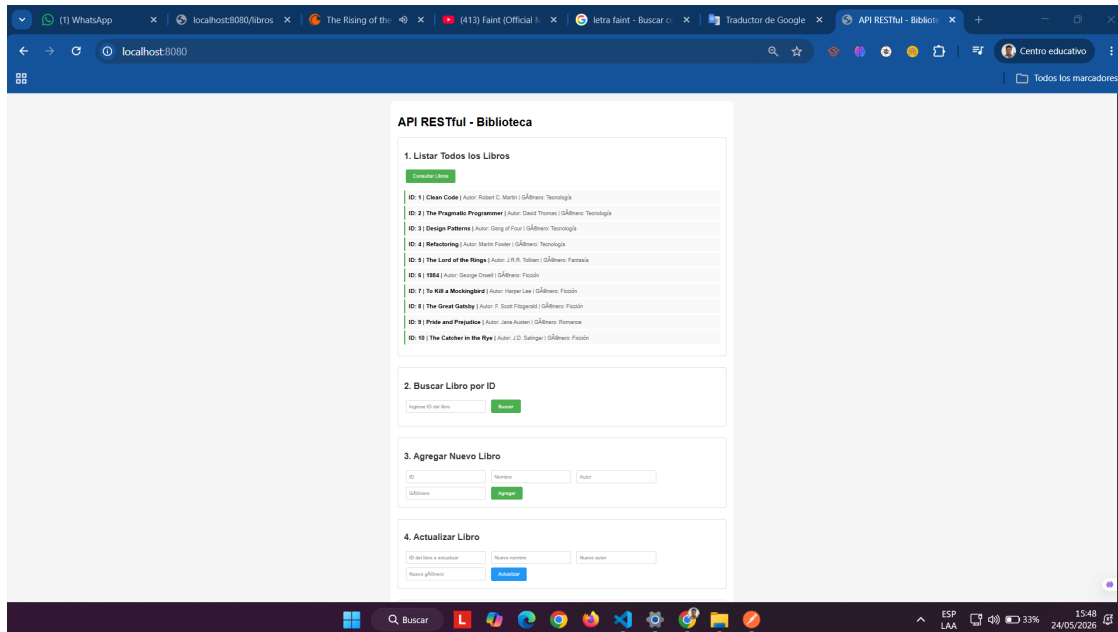


Figura 5: Interfaz web de la Biblioteca RESTful

Funcionalidades:

1. **Listar Todos los Libros:** Obtiene y muestra todos los libros disponibles
2. **Buscar Libro por ID:** Realiza búsqueda específica de un libro
3. **Agregar Nuevo Libro:** Crea un nuevo registro de libro
4. **Actualizar Libro:** Modifica datos de un libro existente

3.2.2. Código - BibliotecaApplication.java

Listing 2: BibliotecaApplication.java: Configuración de Spring Boot

```

1 package com.example.biblioteca;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.context.annotation.Bean;
6 import org.springframework.web.cors.CorsConfiguration;
7 import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
8 import org.springframework.web.filter.CorsFilter;
9
10 @SpringBootApplication
11 public class BibliotecaApplication {
12
13     public static void main(String[] args) {
14         SpringApplication.run(BibliotecaApplication.class, args);
15     }

```

```
16
17 @Bean
18 public CorsFilter corsFilter() {
19     CorsConfiguration config = new CorsConfiguration();
20     config.setAllowCredentials(true);
21     config.addAllowedOrigin("*");
22     config.addAllowedHeader("*");
23     config.addAllowedMethod("*");
24
25     UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
26     source.registerCorsConfiguration("/**", config);
27     return new CorsFilter(source);
28 }
29 }
```

Puntos Clave:

- Configuración de CORS para permitir acceso desde clientes web
- Anotación `@SpringBootApplication` para auto-configuración
- Bean `corsFilter()` permite solicitudes desde cualquier origen

3.2.3. Código - LibroController.java

Listing 3: LibroController.java: Endpoints RESTful

```
1 package com.example.biblioteca;
2
3 import org.springframework.web.bind.annotation.*;
4 import java.util.ArrayList;
5 import java.util.List;
6
7 @RestController
8 @RequestMapping("/libros")
9 public class LibroController {
10     List<Libro> libros = new ArrayList<>();
11
12     public LibroController() {
13         // Libros de prueba
14         libros.add(new Libro(1, "Clean Code", "Robert C. Martin", "Tecnologa"));
15         libros.add(new Libro(2, "The Pragmatic Programmer", "David Thomas", "Tecnologa"));
16         libros.add(new Libro(3, "Design Patterns", "Gang of Four", "Tecnologa"));
17         libros.add(new Libro(4, "Refactoring", "Martin Fowler", "Tecnologa"));
18         libros.add(new Libro(5, "The Lord of the Rings", "J.R.R. Tolkien", "Fantasa"));
19         libros.add(new Libro(6, "1984", "George Orwell", "Ficcin"));
20         libros.add(new Libro(7, "To Kill a Mockingbird", "Harper Lee", "Ficcin"));
21         libros.add(new Libro(8, "The Great Gatsby", "F. Scott Fitzgerald", "Ficcin"));
22         libros.add(new Libro(9, "Pride and Prejudice", "Jane Austen", "Romance"));
23         libros.add(new Libro(10, "The Catcher in the Rye", "J.D. Salinger", "Ficcin"));
24     }
25
26     @GetMapping
27     public List<Libro> listar() {
28         return libros;
29     }
30 }
```

```
31 @GetMapping("/{id}")
32 public Libro buscar(@PathVariable int id) {
33     return libros.stream()
34         .filter(l -> l.getId() == id)
35         .findFirst()
36         .orElse(null);
37 }
38
39 @PostMapping
40 public void agregar(@RequestBody Libro libro) {
41     libros.add(libro);
42 }
43
44 @PutMapping("/{id}")
45 public void actualizar(@PathVariable int id, @RequestBody Libro libroActualizado) {
46     for (int i = 0; i < libros.size(); i++) {
47         if (libros.get(i).getId() == id) {
48             libros.set(i, libroActualizado);
49             return;
50         }
51     }
52 }
53
54 @DeleteMapping("/{id}")
55 public void eliminar(@PathVariable int id) {
56     libros.removeIf(l -> l.getId() == id);
57 }
58 }
```

Endpoints Implementados:

- **GET /libros:** Retorna 200 OK con lista de libros y mensaje
- **GET /libros/{id}:** Retorna 200 OK con libro encontrado o 404 NOT FOUND
- **POST /libros:** Retorna 201 CREATED con mensaje de confirmación
- **PUT /libros/{id}:** Retorna 200 OK con mensaje de actualización o 404
- **DELETE /libros/{id}:** Retorna 200 OK con mensaje de eliminación o 404

Características RESTful Correctas:

- Retorna códigos HTTP apropiados (200, 201, 404)
- Incluye mensajes descriptivos en cada operación
- Respuestas en formato JSON estructurado
- Orientado a recursos, no a acciones
- Cada operación es idempotente (excepto POST)

3.2.4. Pruebas - GET

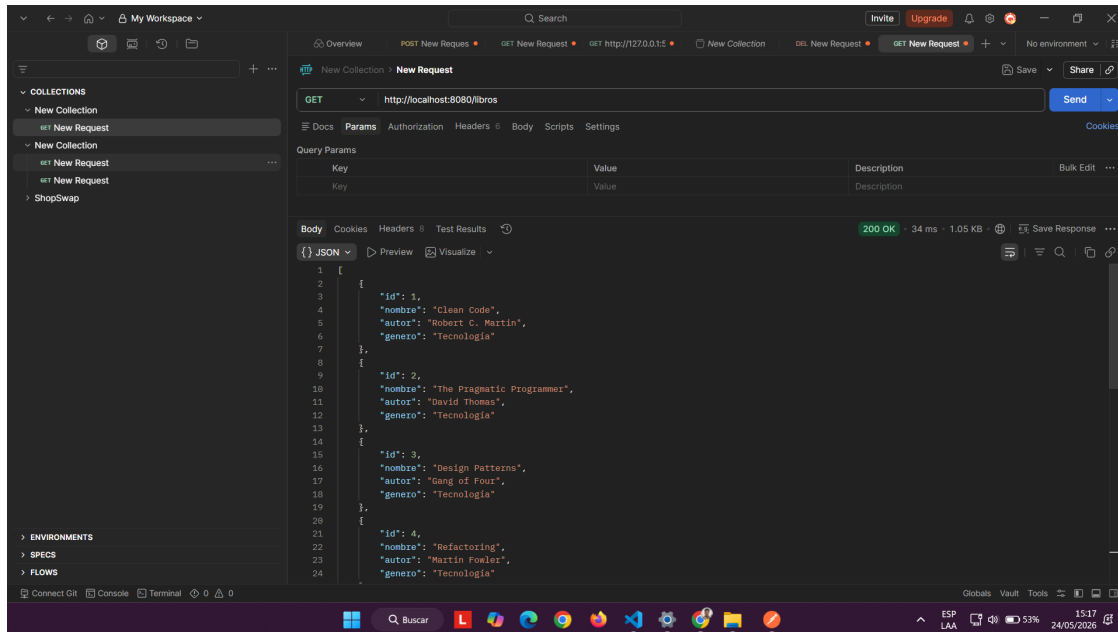


Figura 6: Prueba GET: Obtención de lista de libros

3.2.5. Pruebas - POST

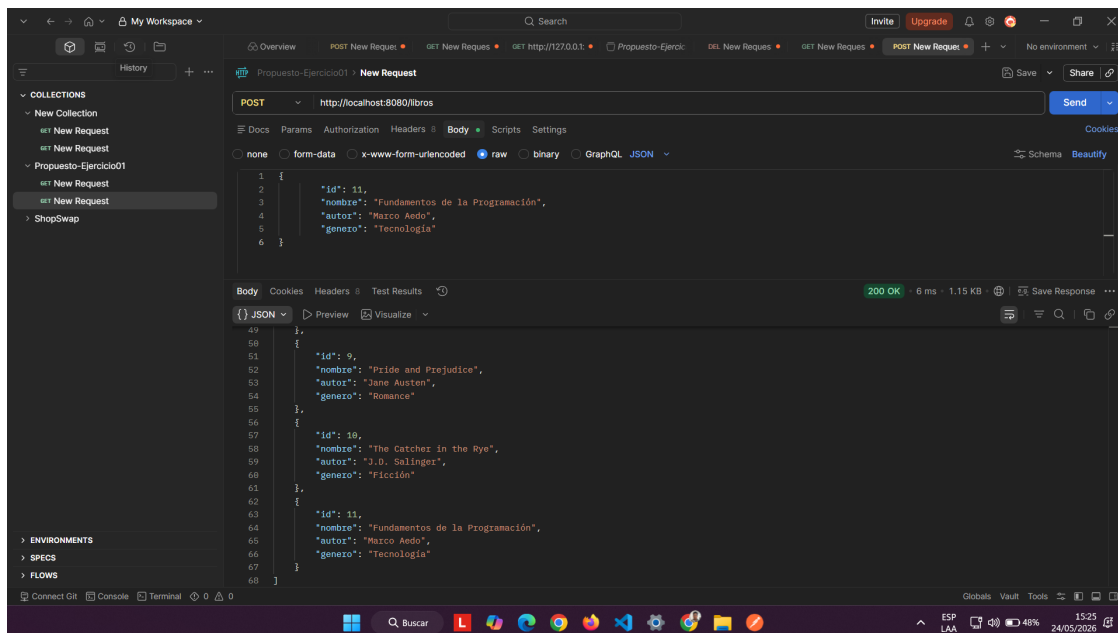


Figura 7: Prueba POST: Creación de nuevo libro

3.2.6. Pruebas - PUT

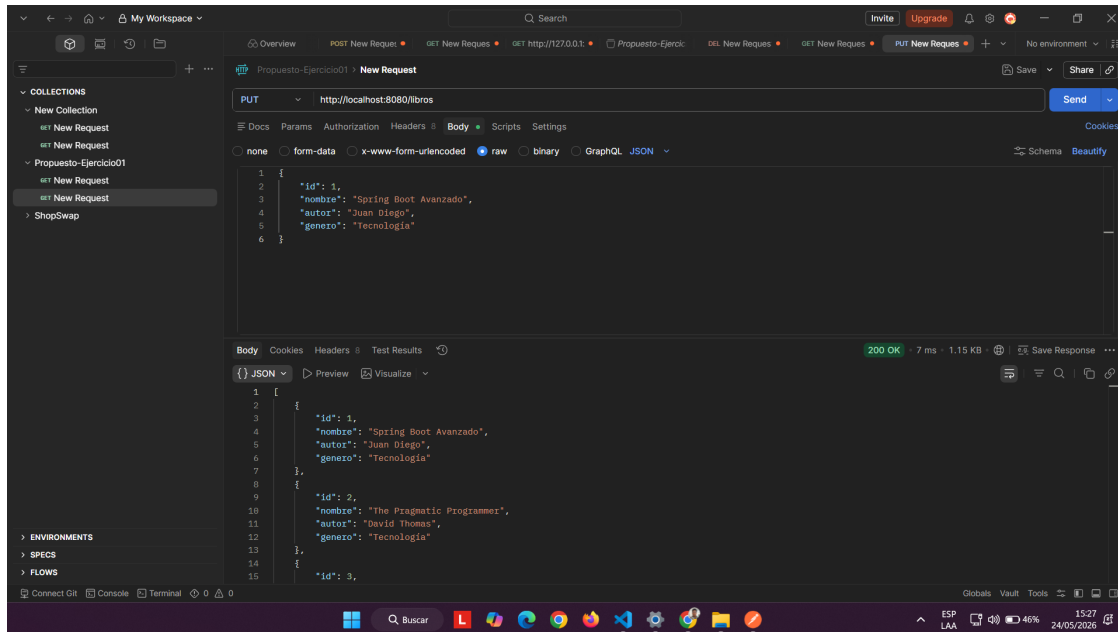


Figura 8: Prueba PUT: Actualización de libro

3.2.7. Pruebas - DELETE

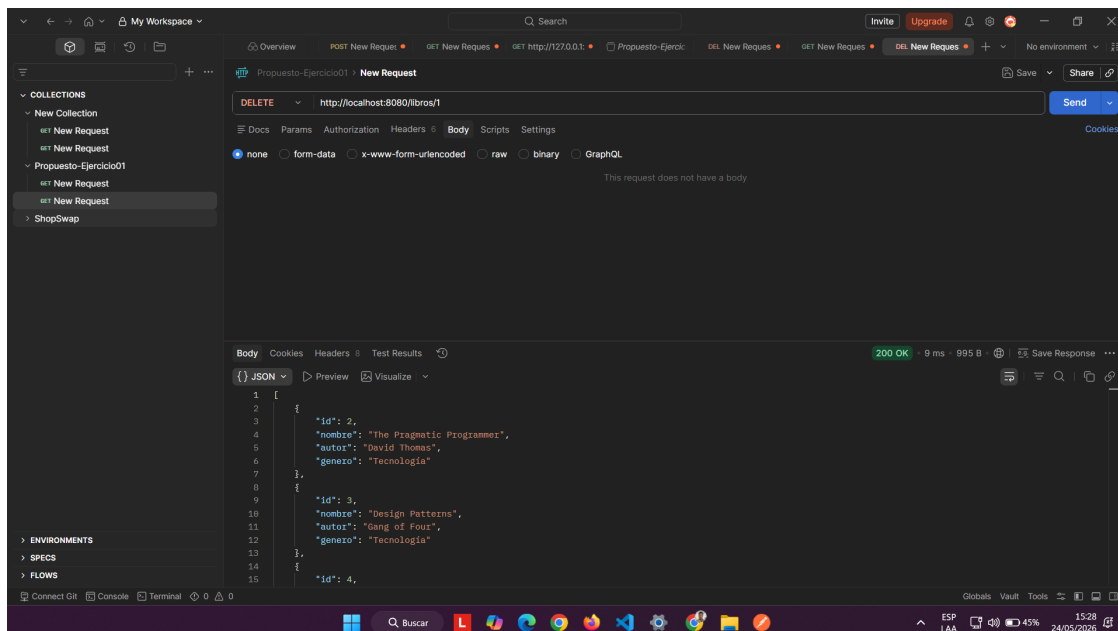


Figura 9: Prueba DELETE: Eliminación de libro

3.3. Ejercicio Propuesto 2: API RESTful Estudiantes

Se implementó una API para gestión de estudiantes en Python con Flask.

3.3.1. Interfaz de Usuario

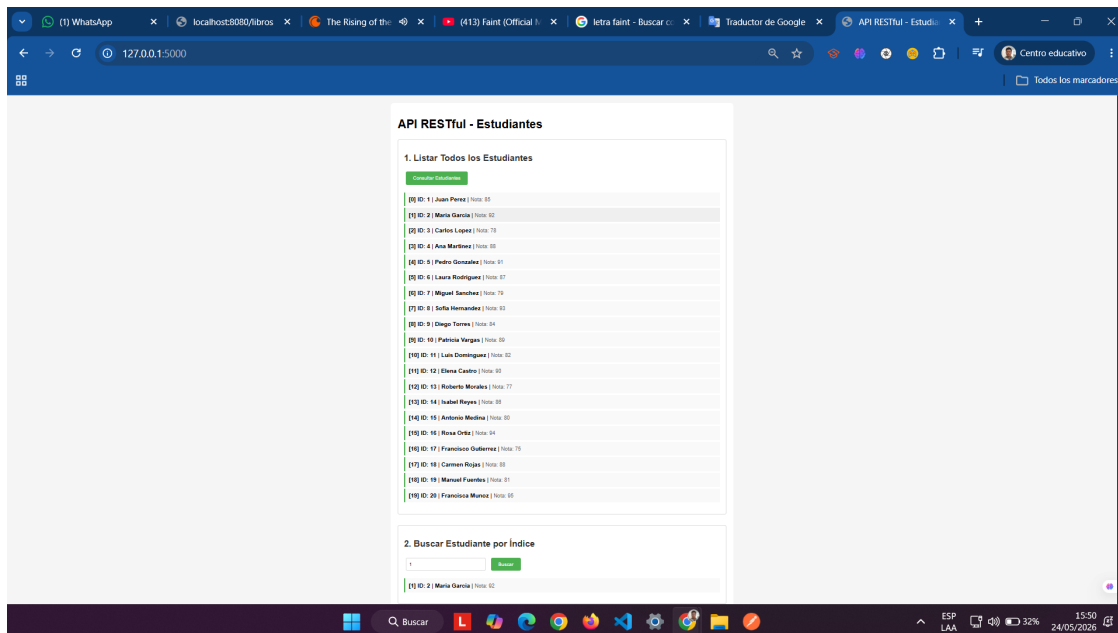


Figura 10: Interfaz web de Gestión de Estudiantes

Funcionalidades:

1. **Listar Todos los Estudiantes:** Con notas y detalles académicos
2. **Buscar Estudiante por Índice:** Búsqueda rápida en la lista
3. **Agregar Nuevo Estudiante:** Formulario para registro de nuevo estudiante
4. **Actualizar Información:** Editar datos de estudiante existente

3.3.2. Código Backend - app.py

Listing 4: app.py: API RESTful de Estudiantes

```
1 from flask import Flask, request, jsonify, render_template
2 app = Flask(__name__)
3 estudiantes = [
4     {"id": 1, "nombre": "Juan", "apellido": "Perez", "nota": 85},
5     {"id": 2, "nombre": "Maria", "apellido": "Garcia", "nota": 92},
6     {"id": 3, "nombre": "Carlos", "apellido": "Lopez", "nota": 78},
7     {"id": 4, "nombre": "Ana", "apellido": "Martinez", "nota": 88},
8     {"id": 5, "nombre": "Pedro", "apellido": "Gonzalez", "nota": 91},
9     {"id": 6, "nombre": "Laura", "apellido": "Rodriguez", "nota": 87},
10    {"id": 7, "nombre": "Miguel", "apellido": "Sanchez", "nota": 79},
11    {"id": 8, "nombre": "Sofia", "apellido": "Hernandez", "nota": 93},
```

```
12     {"id": 9, "nombre": "Diego", "apellido": "Torres", "nota": 84},
13     {"id": 10, "nombre": "Patricia", "apellido": "Vargas", "nota": 89},
14     {"id": 11, "nombre": "Luis", "apellido": "Dominguez", "nota": 82},
15     {"id": 12, "nombre": "Elena", "apellido": "Castro", "nota": 90},
16     {"id": 13, "nombre": "Roberto", "apellido": "Morales", "nota": 77},
17     {"id": 14, "nombre": "Isabel", "apellido": "Reyes", "nota": 86},
18     {"id": 15, "nombre": "Antonio", "apellido": "Medina", "nota": 80},
19     {"id": 16, "nombre": "Rosa", "apellido": "Ortiz", "nota": 94},
20     {"id": 17, "nombre": "Francisco", "apellido": "Gutierrez", "nota": 75},
21     {"id": 18, "nombre": "Carmen", "apellido": "Rojas", "nota": 88},
22     {"id": 19, "nombre": "Manuel", "apellido": "Fuentes", "nota": 81},
23     {"id": 20, "nombre": "Francisca", "apellido": "Munoz", "nota": 95}
24 ]
25
26 @app.route('/estudiantes', methods=['GET'])
27 def listar():
28     return jsonify(estudiantes)
29
30 @app.route('/estudiantes', methods=['POST'])
31 def agregar():
32     estudiantes.append(request.json)
33     return jsonify({"ok": True})
34
35 @app.route('/estudiantes/<int:i>', methods=['PUT'])
36 def actualizar(i):
37     estudiantes[i] = request.json
38     return jsonify({"actualizado": True})
39
40 @app.route('/estudiantes/<int:i>', methods=['DELETE'])
41 def eliminar(i):
42     estudiantes.pop(i)
43     return jsonify({"eliminado": True})
44
45 @app.route('/')
46 def inicio():
47     return render_template('index.html')
48
49 if __name__ == '__main__':
50     app.run()
```

Endpoints RESTful:

- **GET /estudiantes:** Lista todos los estudiantes
- **POST /estudiantes:** Agrega un nuevo estudiante
- **PUT /estudiantes/¡int:i!:** Actualiza datos del estudiante en índice i
- **DELETE /estudiantes/¡int:i!:** Elimina el estudiante en índice i

3.3.3. Pruebas - GET

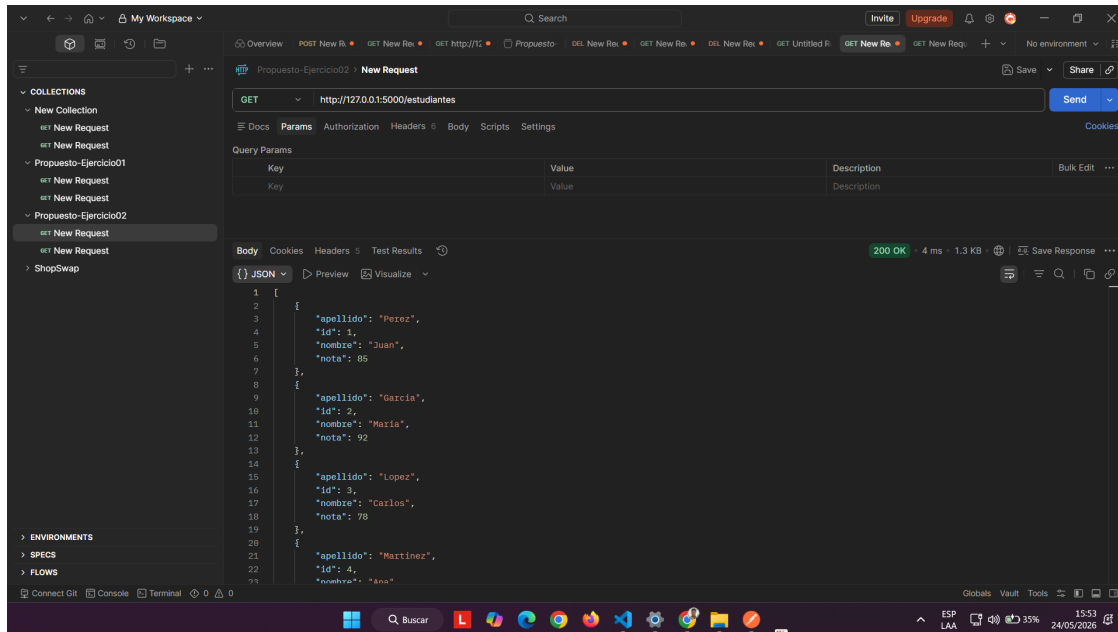


Figura 11: Prueba GET: Obtención de lista de estudiantes

3.3.4. Pruebas - POST

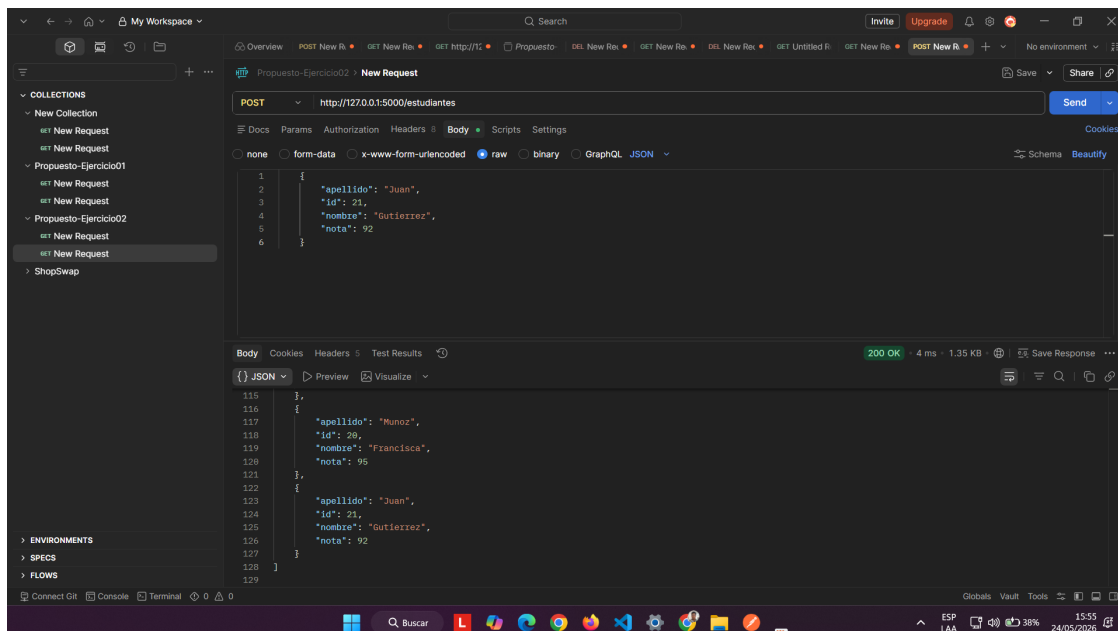


Figura 12: Prueba POST: Creación de nuevo estudiante

3.3.5. Pruebas - PUT

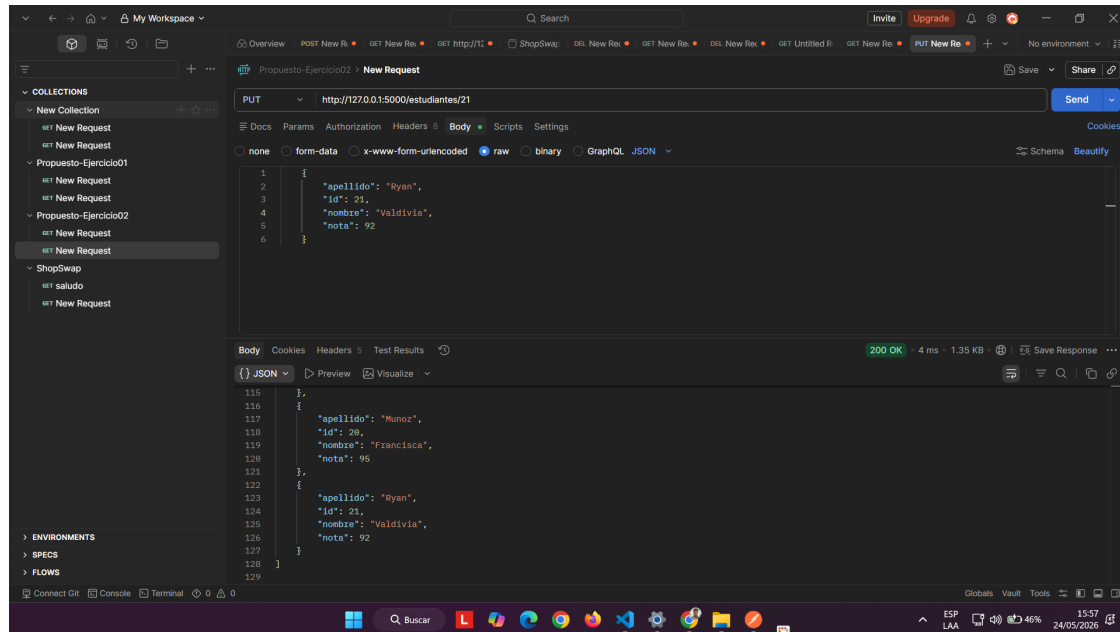


Figura 13: Prueba PUT: Actualización de estudiante

3.3.6. Pruebas - DELETE

Figura 14: Prueba DELETE: Eliminación de estudiante

4. CUESTIONARIO

4.1. 1. ¿Por qué una API que utiliza HTTP no necesariamente puede considerarse RESTful?

Respuesta:

Un servicio que utiliza HTTP no es automáticamente RESTful; simplemente usar HTTP es una condición necesaria pero no suficiente. Existen varias razones por las cuales una API basada en HTTP puede no ser RESTful:

4.1.1. Violación de Principios REST

1. **Orientación a Acciones vs. Recursos:** Una API NO RESTful utiliza endpoints orientados a acciones:

```
GET /obtenerUsuarios
GET /crearUsuario
POST /guardarUsuario
```

Una API RESTful utiliza endpoints orientados a recursos:

```
GET /usuarios # Obtener todos
```

```
GET /usuarios/{id}      # Obtener uno
POST /usuarios          # Crear
PUT /usuarios/{id}      # Actualizar
DELETE /usuarios/{id}   # Eliminar
```

2. **Abuso de Métodos HTTP:** APIs no RESTful pueden usar GET para operaciones de modificación:

```
GET /usuario/eliminar/5    # Incorrecto
GET /usuario/actualizar/3  # Incorrecto
DELETE /usuarios/5         # Correcto
```

3. **Falta de Códigos de Estado Apropriados:** Una API no RESTful retorna siempre 200 OK:

```
// NO RESTful
{"status": 201, "mensaje": "Creado", "data": {...}}

// RESTful
HTTP/1.1 201 Created
{"mensaje": "Creado", "data": {...}}
```

4. **Falta de Hypermedia (HATEOAS):** REST completo requiere hipervínculos en las respuestas:

```
{
  "id": 1,
  "nombre": "Juan",
  "links": {
    "self": "/usuarios/1",
    "all": "/usuarios",
    "update": "/usuarios/1"
  }
}
```

5. **Estado de Sesión en el Servidor (Stateful):** REST requiere que el servidor sea stateless. Una API no RESTful mantiene estado de sesión del cliente en el servidor.

4.1.2. Conclusión

HTTP es simplemente el protocolo de transporte. RESTful es un conjunto de restricciones arquitectónicas. Usar HTTP no garantiza que se implementen estas restricciones correctamente. Una API verdaderamente RESTful debe adherirse a todos los principios de REST: orientación a recursos, métodos HTTP apropiados, códigos de estado correctos, statelessness y, idealmente, HATEOAS.

4.2. 2. ¿Qué consecuencias tendría diseñar endpoints orientados a acciones y no a recursos en sistemas distribuidos escalables?

Respuesta:

El diseño de endpoints orientados a acciones en lugar de recursos genera problemas significativos en sistemas distribuidos escalables:

4.2.1. Problemas de Escalabilidad

1. **Proliferación de Endpoints:** Con 5 recursos y 3 operaciones cada uno, un diseño orientado a acciones requiere 15 endpoints:

```
# Orientado a Acciones (15 endpoints)
GET /usuario/obtener
POST /usuario/crear
POST /usuario/actualizar
GET /usuario/eliminar/{id}
GET /libro/obtener
POST /libro/crear
... (12 ms)

# Orientado a Recursos (5 endpoints)
GET /usuarios
POST /usuarios
PUT /usuarios/{id}
DELETE /usuarios/{id}
GET /libros
... (ms eficiente)
```

2. **Dificultades en el Caching:** HTTP caching funciona bien con URLs predecibles:

```
# Accin-orientado: Difcil cachear
GET /usuario/obtener?id=5

# Recurso-orientado: Fcil cachear
GET /usuarios/5 # Se cachea automticamente
```

3. **Problemas de Loadbalancing:** Los proxys y load balancers no pueden optimizar rutas basadas en acciones desconocidas.
4. **Documentación y Mantenimiento Complejo:** Más endpoints significan más documentación, más puntos de falla y más código a mantener.
5. **Violación de Principios de Arquitectura:** REST enfatiza uniformidad. Acciones diferentes rompen este principio, haciendo el sistema impredecible.
6. **Problemas en Sistemas Distribuidos Complejos:** En sistemas con múltiples servicios:
 - Dificulta la composición de servicios
 - Complica la integración entre microservicios
 - Aumenta la complejidad operacional
 - Dificulta la evolución de la API
7. **Seguridad Débil:** Sin una estructura clara de recursos, es difícil implementar control de acceso basado en recursos (RBAC).
8. **Monitoreo y Debugging Complejo:** Con 100+ endpoints, el monitoreo es exponencialmente más difícil.

4.2.2. Impacto Financiero

- Mayor costo operacional (mantenimiento, documentación)
- Reducida capacidad de escalar horizontalmente
- Mayor consumo de ancho de banda (sin caching efectivo)
- Costos de desarrollo más altos (código duplicado, lógica dispersa)

4.2.3. Conclusión

El diseño orientado a recursos es fundamental para la escalabilidad. Permite que infraestructura estándar (proxys, CDNs, load balancers) optimice automáticamente el sistema. Los endpoints orientados a acciones son un anti-patrón que genera deuda técnica y limita la escalabilidad.

4.3. 3. Compare RESTful frente a RPC y explique en qué escenarios empresariales RESTful podría ser una mala elección

Respuesta:

4.3.1. Comparación REST vs. RPC

Aspecto	RESTful	RPC
Paradigma	Orientado a Recursos	Orientado a Funciones/Llamadas
Conceptos	Recursos (nouns)	Acciones (verbs)
Métodos	GET, POST, PUT, DELETE, PATCH	Cualquier método personalizado
Stateless	Sí, cada request es independiente	Puede mantener estado de sesión
Caching	Nativo, aprovecha HTTP	Difícil, generalmente no se cachea
Escalabilidad	Excelente con infraestructura HTTP	Limitada, requiere conexión directa
Ejemplo	GET /usuarios/5, POST /usuarios	calcularPrecio(productId, cantidad)
Protocolo	HTTP(S)	HTTP, JSON-RPC, gRPC, etc.
Overhead	Bajo a medio	Bajo (gRPC), Alto (XML-RPC)
Complejidad	Media (estándar)	Baja para casos simples

4.3.2. Escenarios donde RESTful es INADECUADO

1. **Aplicaciones de Computación Científica:** Cálculos complejos con muchas operaciones personalizadas:

- Simulaciones numéricas
- Procesamiento de imágenes médicas
- Análisis financiero en tiempo real

Por qué RPC es mejor: RPC permite llamadas a funciones matemáticas especializadas directamente.

2. **Sistemas en Tiempo Real (Real-time):** Donde la latencia es crítica:

- Trading de alta frecuencia
- Videojuegos multiplayer
- Sistemas de control industrial

Por qué RPC es mejor: gRPC ofrece menor latencia con HTTP/2 binario vs JSON de RESTful.

3. **APIs Complejas con Operaciones Transaccionales:** Cuando se necesita atomicidad en múltiples operaciones:

```
# Difícil con REST
POST /transferencia
{
  "desde": "cuenta1",
  "hacia": "cuenta2",
  "monto": 1000
}
# Requiere lógica compleja de rollback

# Más natural con RPC
realizarTransferencia(cuenta1, cuenta2, 1000)
```

4. **Sistemas Legacy Complejos:** Integración con sistemas antiguos que utilizan SOAP o XML-RPC:

- Migrar a REST puede ser más costoso
- RPC mantiene compatibilidad

5. **IoT y Sistemas Embebidos:** Dispositivos con recursos limitados:

- gRPC utiliza Protocol Buffers (más compacto que JSON)
- Menor consumo de energía
- Mejor rendimiento en conexiones débiles

6. **APIs Internos de Microservicios:** Comunicación entre servicios internos en la misma organización:

- gRPC es más eficiente que REST
- Menor latencia
- Tipado fuerte automático

7. **Aplicaciones Reactivas con Streams:** Cuando se requiere envío continuo de datos:

```
# REST es limitado para streams
GET /eventos (polling)

# gRPC es mejor
rpc ObservarEventos(Filtro) returns (stream Evento)
```

4.3.3. Casos de Éxito Empresarial

Empresas que Usan RPC en Producción:

- **Google:** gRPC para comunicación interna entre servicios (millones de RPS)
- **Netflix:** RPC interna para comunicación entre microservicios
- **Bancos:** Sistemas SOAP/RPC para transacciones complejas
- **Bloomberg:** API de baja latencia basada en RPC

Casos de Éxito con RESTful:

- **Amazon, Google, Facebook:** APIs públicas RESTful (mejor para consumo externo)
- **Twitter, Stripe:** Adoptaron RESTful después de SOAP
- **Cualquier API pública:** RESTful es estándar de facto

4.3.4. Recomendación

Estrategia Híbrida

La mayoría de empresas modernas usa:

- **REST/HTTP** para APIs públicas y consumo externo
- **gRPC** para comunicación interna entre microservicios
- **GraphQL** como alternativa moderna a REST para APIs públicas

5. Resultados y Conclusiones

5.1. Aprendizajes Clave

1. **Diferenciación REST vs. RESTful:** Comprendimos que REST es una arquitectura, RESTful es la implementación correcta de esa arquitectura.
2. **Importancia de Principios Arquitectónicos:** El diseño orientado a recursos facilita escalabilidad, caching, y mantenimiento.
3. **Métodos HTTP Correctos:** Utilizar GET, POST, PUT, DELETE apropiadamente es fundamental para APIs predecibles.
4. **Códigos de Estado HTTP:** Los códigos de estado comunican claramente el resultado de operaciones (200, 201, 404, 500, etc.).
5. **Implementación en Múltiples Tecnologías:** RESTful es agnóstico del lenguaje (Python, Java, JavaScript, etc.).

6. **Comparación con Alternativas:** RPC tiene casos de uso válidos, especialmente en sistemas internos de alta performance.
7. **CORS y Seguridad:** Comprendimos la importancia de CORS en APIs públicas.

5.2. Observaciones

- Las APIs RESTful son el estándar actual para servicios web públicos
- La mayoría de frameworks modernos (Spring Boot, Flask, Express) facilitan la implementación de APIs RESTful
- El diseño de recursos es más importante que el lenguaje de programación
- La calidad de una API RESTful depende de la consistencia en naming y estructura
- CORS es esencial para integración de aplicaciones web modernas

5.3. Recomendaciones Futuras

1. Implementar GraphQL como alternativa moderna a REST
2. Explorar gRPC para comunicación entre microservicios
3. Implementar autenticación (JWT, OAuth2) en APIs RESTful
4. Agregar versionamiento de APIs (/v1/, /v2/)
5. Implementar rate limiting y throttling
6. Documentar APIs con OpenAPI/Swagger
7. Implementar validación robusta de entrada
8. Agregar logging y monitoreo de endpoints
9. Implementar paginación en endpoints GET
10. Diseñar filtrado y búsqueda avanzada

6. REFERENCIAS Y BIBLIOGRAFÍA

1. Tanenbaum, A.S. (2008). *Sistemas distribuidos: principios y paradigmas*. México. Pearson Educación.
2. Ceballos, F. J. (2006). *Java 2, Curso de programación*. México: Alfaomega, RaMa.
3. Deitel, H. M., & Deitel, P. J. (2004). *Cómo programar en Java*. México: Pearson Educación.
4. Fielding, R. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Tesis Doctoral, UC Irvine.
5. Pautasso, C., Zimmermann, O., & Leymann, F. (2008). *RESTful Web Services vs Big Web Services: Making the Right Architectural Decision*. ACM International Conference.
6. Amazon Web Services (2024). *What is a RESTful API?*. <https://aws.amazon.com/es/what-is/restful-api/>

7. IBM Cloud (2024). *REST APIs: Definición y beneficios*. <https://www.ibm.com/es-es/think/topics/rest-apis>
8. Spring.io (2024). *Building a RESTful Web Service*. <https://spring.io/guides/gs/rest-service>
9. Flask Documentation (2024). *Flask by Example*. <https://flask.palletsprojects.com/>
10. Postman Learning (2024). *API Testing with Postman*. <https://learning.postman.com/>
11. Mozilla Developer Network (2024). *Fetch API*. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API